A.Nandini
2303A51512
B-26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week4 – Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
AssignmentNumber:7.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1		Week4 -

Wednesday

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Lab Objectives

• To identify and correct syntax, logic, and runtime errors in Python programs using AI tools
• To understand common programming bugs and AI-assisted debugging suggestions
• To evaluate how AI explains, detects, and fixes different types of coding errors
• To build confidence in using AI for structured debugging practices

Lab Outcomes (LOs)

After completing this lab, students will be able to:
• Use AI tools to detect and correct syntax, logic, and runtime errors
• Interpret AI-suggested bug fixes and explanations
• Apply systematic debugging strategies using AI-generated insights
• Refactor buggy code using reliable programming patterns

Task 1: Fixing Syntax Errors

Scenario

You are reviewing a Python program where a basic function definition contains a syntax error.

```python
def add(a, b)
    return a + b
```

Requirements

• Provide a Python function add(a, b) with a **missing colon**
• Use an AI tool to detect the syntax error
• Allow AI to correct the function definition
• Observe how AI explains the syntax issue

Expected Output

• Corrected function with proper syntax
• Syntax error resolved successfully
• AI-generated explanation of the fix

PROMPT: Here is a Python function with a syntax error. Please find the error, fix it, and explain what was wrong. def add(a, b) return a + b

EXPLAINATION: The error was caused by a missing colon (:) at the end of the function definition.
In Python, a colon is required to start the function body.
Adding it fixes the syntax and allows the code to run properly.

Task 2: Debugging Logic Errors in Loops

Scenario
You are debugging a loop that runs infinitely due to a logical mistake.

```python
def count_down(n):
    while n >= 0:
        print(n)
        n += 1  # Should be n -= 1
```

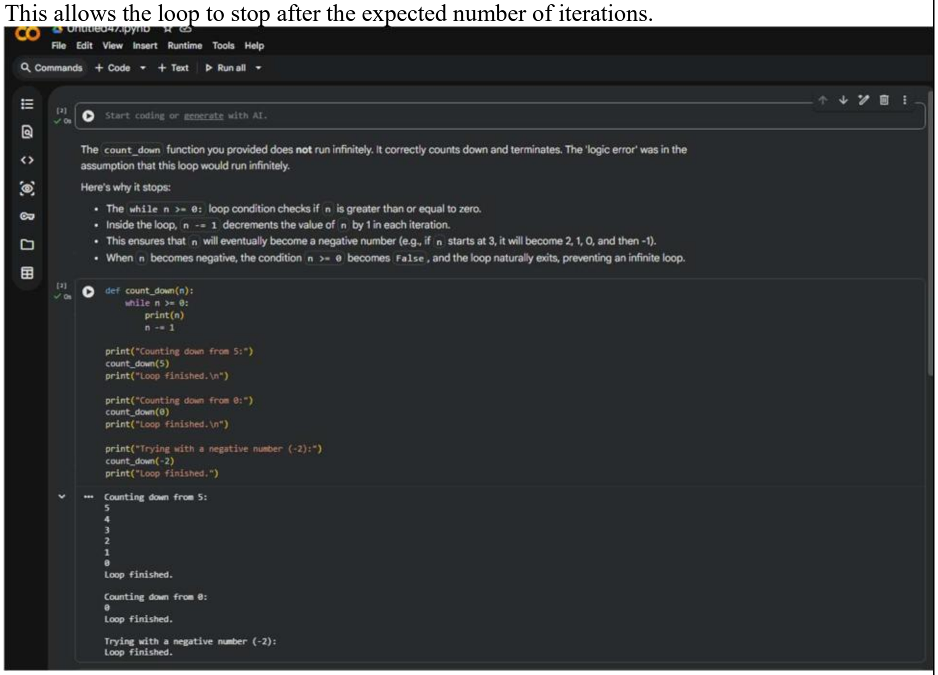
Requirements
• Provide a loop with an **increment or decrement error**
• Use AI to identify the cause of infinite iteration
• Let AI fix the loop logic
• Analyze the corrected loop behavior

Expected Output
• Infinite loop issue resolved
• Correct increment/decrement logic applied
• AI explanation of the logic error

PROMPT: Here is a loop that runs infinitely. Please identify why it does not stop, fix the logic error, and explain the correction.

```
def count down(n):
    while n >= 0:
        print(n)
        n -= 1
```

EXPLAINATION: The loop was infinite because the counter variable was never updated correctly inside the loop.
By adding the proper increment/decrement, the condition eventually becomes false.
This allows the loop to stop after the expected number of iterations.



Task 3: Handling Runtime Errors (Division by Zero)

Scenario
A Python function crashes during execution due to a division by zero error.

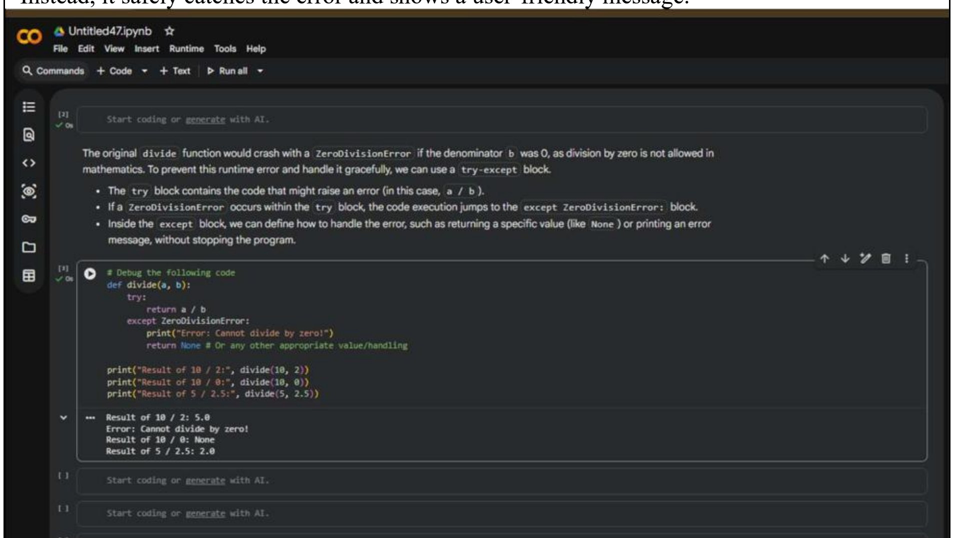
```python
# Debug the following code
def divide(a, b):
    return a / b

print(divide(10, 0))
```

Requirements
• Provide a function that performs division without validation
• Use AI to identify the runtime error
• Let AI add try-except blocks for safe execution
• Review AI's error-handling approach

Expected Output
• Function executes safely without crashing
• Division by zero handled using try-except
• Clear AI-generated explanation of runtime error handling

PROMPT: Here is a Python function that performs division but crashes at runtime. Please identify the error, fix it using try-except, and explain how the error is handled."
EXPLAINATION: The program crashes when division by zero occurs, which raises a runtime error.
Using a try-except block prevents the program from stopping suddenly.
Instead, it safely catches the error and shows a user-friendly message.



Task 4: Debugging Class Definition Errors

Scenario
You are given a faulty Python class where the constructor is incorrectly defined.

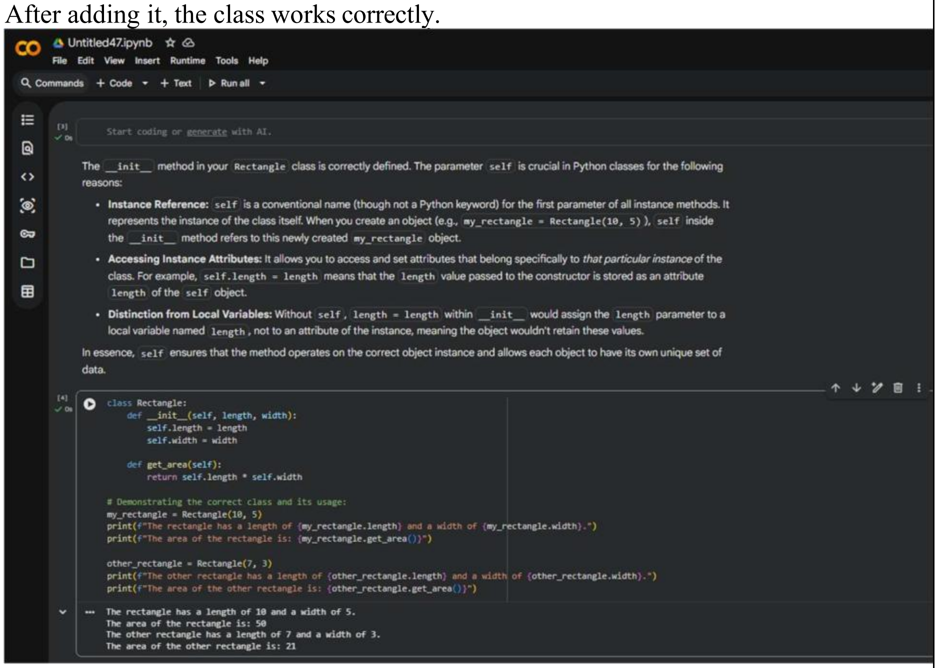
```python
class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Requirements

• Provide a class definition with **missing self-parameter**
• Use AI to identify the issue in the __init__() method
• Allow AI to correct the class definition
• Understand why self is required

Expected Output
• Corrected __init__() method
• Proper use of self in class definition
• AI explanation of object-oriented error

PROMPT: Here is a Python class with an incorrect constructor. Please find the error in the __init method, fix it, and explain why self is required.
EXPLAINATION: The constructor was missing self as the first parameter.
self is required to store values inside the object.
After adding it, the class works correctly.



Task 5: Resolving Index Errors in Lists

Scenario
A program crashes when accessing an invalid index in a list.

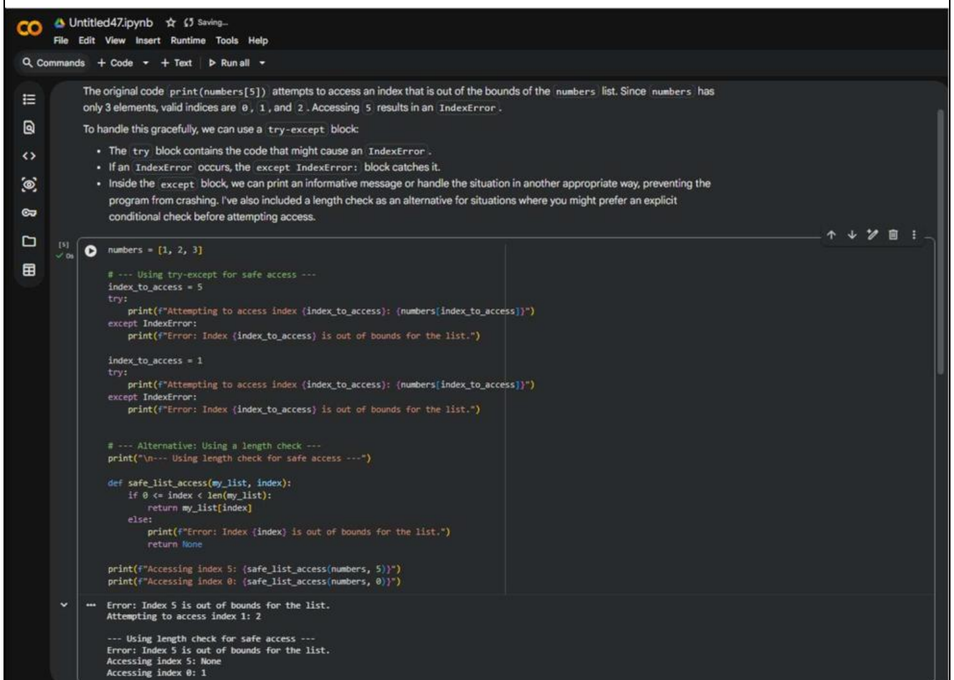
```python
numbers = [1, 2, 3]
print(numbers[5])
```

Requirements
• Provide code that accesses an **out-of-range list index**
• Use AI to identify the Index Error
• Let AI suggest safe access methods
• Apply bounds checking or exception handling

Expected Output
• Index error resolved
• Safe list access logic implemented
• AI suggestion using length checks or exception handling.

PROMPT: Here is a Python program that crashes due to an invalid list index. Please identify the IndexError, fix it using safe list access (length check or try-except), and explain the solution.
numbers = [1, 2, 3]
print(numbers[5])

EXPLAINATION: The error occurs because the program tries to access an index that does not exist in the list.
By checking the list length or using try-except, we prevent the crash.
This makes the program run safely even when the index is invalid.



Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots